

Jessica Mosheev_Terraform_EXAM

Section 1: Q&A (20 Questions)

- Terraform Fundamentals (5 questions)

- What is Terraform and how does it differ from other IaC tools?

Infrastructure as Code (IaC) tools allow you to manage infrastructure with configuration files rather than through a graphical user interface. IaC allows you to build, change, and manage your infrastructure in a safe, consistent, and repeatable way by defining resource configurations that you can version, reuse, and share.

Terraform is HashiCorp's infrastructure as code tool. It lets you define resources and infrastructure in human-readable, declarative configuration files, and manages your infrastructure's lifecycle. Using Terraform has several advantages over manually managing your infrastructure:

- Terraform can manage infrastructure on multiple cloud platforms.
- The human-readable configuration language helps you write infrastructure code quickly.
- Terraform's state allows you to track resource changes throughout your deployments.
- You can commit your configurations to version control to safely collaborate on infrastructure.

- Explain Terraform's declarative nature and state management.

Terraform is a declarative Infrastructure as Code (IaC) tool, meaning you describe the desired state of your infrastructure, and Terraform figures out the necessary steps to achieve it. This simplifies the process, as you focus on defining outcomes rather than procedures.

It also uses a state file to track the current status of your infrastructure. This file helps Terraform map resources, detect changes, and generate efficient plans to apply only required updates. Managing state efficiently allows Terraform to handle resource dependencies and support collaborative work with remote state storage solutions.

- What is the purpose of the Terraform provider?

A provider in Terraform is a plugin that allows Terraform to interact with APIs of external platforms, such as cloud services or SaaS tools. Providers manage the lifecycle of the resources they expose, handling actions like creation, updates, and deletion.

In your Terraform configuration, you define which providers to use and configure any required credentials or settings. This tells Terraform what services it should manage and how to connect to them.

- How does Terraform handle dependency resolution?

Terraform creates resources in parallel and can automatically handle resource dependencies using implicit relationships. These dependencies are inferred when one resource references another, such as a subnet depending on a VPC by referencing its id. This allows Terraform to determine the correct order of operations without manual intervention.

However, when no clear relationship exists in the configuration, Terraform cannot automatically resolve the order.

- What are the key components of a Terraform configuration file?

Providers define the external services that Terraform will interact with, such as AWS, Azure, or Google Cloud. They tell Terraform where and how to manage resources.

Resources define the actual infrastructure components, like instances, databases, or networks. These are the items Terraform will create, update, or delete.

Variables allow you to define dynamic values that can be reused across the configuration. They make your setup more flexible and reusable.

Outputs provide information about the infrastructure after it's been created. These can be things like instance IDs, IP addresses, or other useful details.

Modules are reusable pieces of configuration that can help organize complex setups. They allow for better code management and can be shared across different projects.

- **State Management & Backend Configuration (3 questions)**

- Explain the difference between `terraform refresh`, `terraform plan`, and `terraform apply`.

The `terraform refresh` command reads the current settings from all managed remote objects and updates the Terraform state to match.

The `terraform plan` command creates an execution plan, which lets you preview the changes that Terraform plans to make to your infrastructure. By default, when Terraform creates a plan it:

The `terraform apply` command executes the actions proposed in a Terraform plan.

- What is the difference between local and remote backends?

Local Backend:

Store the state file on your local machine. It's simple and works well for personal or small projects, but not ideal for team collaboration.

Remote Backend:

Stores the state file remotely (e.g., AWS S3, Terraform Cloud), allowing multiple users to share and manage the state. It offers features like state locking and versioning for better collaboration and safety.

- How can you prevent state corruption when multiple engineers work on the same infrastructure?

To prevent state corruption when multiple engineers are working on the same infrastructure, use a remote backend with state locking. This ensures only one person can make changes to the state at a time, preventing conflicts. Additionally, use versioning to roll back changes if needed.

- **Terraform Modules & Reusability (4 questions)**

- What are the benefits of using Terraform modules?

Reusability- Modules allow you to organize infrastructure resources and configurations into containers you can repurpose across projects and environments.

Abstraction: Modules simplify resource creation and configuration processes for Terraform configuration files, making them more concise and understandable.

Encapsulation: Modules isolate resources and their dependencies, making it more straightforward for you to manage or modify individual pieces of your infrastructure without impacting others or hindering modularity in its codebase.

Versioning: Terraform modules can be versioned, making it easier to track changes and update dependencies in an orderly manner.

Collaboration: Modules allow your team and the wider community to work more collaboratively by sharing them.

- Explain how to pass variables to a Terraform module.

First, I define this files in module library:

main.tf: This file defines the resources that are created by the module.

variables.tf: This file defines the variables that can be used to customize the module.

outputs.tf: This file defines the outputs that are produced by the module.

To pass variables to a Terraform module, you first define the variables within the module using a `variables.tf` file, specifying the names, types, and optional default values. When calling the module in your root configuration (typically in `main.tf`), you assign specific values to those variables, customizing the behavior of the module as needed. You can also store variable values separately in a `terraform.tfvars` file to keep your configuration clean and organized. Additionally, the module can provide outputs using an `outputs.tf` file, allowing you to access important information about the resources it creates. This structure makes your infrastructure definitions flexible and reusable.

- What is the difference between `count` and `for_each`?

The main difference is the way they handle resource creation. I use `count` when I need a simple fixed number of identical resources, and I use `for_each` when dealing with collections that require unique handling.

- How do you source a module from a Git repository?

I use the `source` argument in the module block and provide the repository url. I can specify a branch, tag, or commit followed by the desired reference. This allows Terraform to fetch the module directly from the repository and use it in your infrastructure setup.

- **Terraform with AWS (4 questions)**

- How do you create an EC2 instance with Terraform?

First, I define the AWS provider and set up the credentials and region. Next, I use the `'aws_instance'` resource to specify key parameters such as the ami, instance type, and security group.

- What are the required fields for defining a VPC in Terraform?

To define a VPC in Terraform, the only required field is `cidr_block`, which sets the range of IP addresses for the VPC in CIDR format.

```
resource "aws_vpc" "main" {  
  
  cidr_block = "10.0.0.0/16"  
  
}
```

- Explain how Terraform manages IAM policies in AWS.

Terraform uses resources like `aws_iam_policy` to define the policy and `aws_iam_policy_attachment` to attach it to users, groups, or roles. You define the policy's permissions in JSON format, and then attach it to the relevant IAM entities to grant them the specified permissions. This allows for consistent and version-controlled management of IAM policies across your infrastructure.

- How do you use Terraform to provision and attach an Elastic Load Balancer?

First, I define the `aws_lb` resource, give details like the load balancer type, subnets, and security groups. Then, I create `aws_lb_target_group` to define the target group, which will include the instances or resources behind the load balancer. Finally, I use `aws_lb_listener` to configure the listener, which handles incoming traffic and forwards it to the target group. This setup allows you to automate the deployment of ELBs and ensure consistent configuration.

- **Debugging & Error Handling (4 questions)**

- What does the `terraform validate` command do?

The `terraform validate` command checks the configuration files for any syntax errors or issues. It ensures everything is set up correctly, but doesn't actually interact with any infrastructure or apply changes. It's a simple way to make sure your code is valid before moving on to commands like `terraform plan` or `terraform apply`.

- How can you debug Terraform errors effectively?

First, checking the error message, as it often provides useful information about what went wrong. I can use the `terraform plan` command to preview the changes and identify issues before applying them. Enabling detailed logging with `TF_LOG=DEBUG` can also help you see more information about the execution process. Additionally, reviewing the Terraform documentation or searching the error message online can help you find solutions or workarounds for common issues.

- What is Terraform's `ignore_changes` lifecycle policy used for?

The `ignore_changes` lifecycle policy in Terraform prevents updates to specific resource attributes. It's useful when you want to keep manual changes or avoid unnecessary updates to certain settings without Terraform modifying them.

- How do you import existing AWS infrastructure into Terraform?

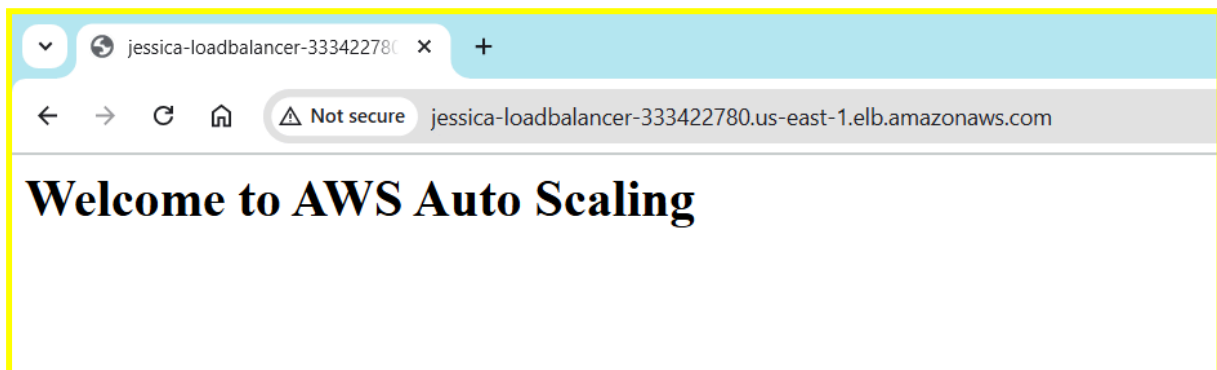
I need to use the `terraform import` command with the resource type and its ID. Before importing, make sure your Terraform configuration matches the existing resource. Once imported, Terraform will recognize it in its state, and you can run `terraform plan` to see if any changes are needed.

Section 2:

<http://jessica-loadbalancer-333422780.us-east-1.elb.amazonaws.com/>

Outputs:

```
alb_output = {
  "alb_dns_name" = "jessica-app-lb-494831708.us-east-1.elb.amazonaws.com"
}
ec2_output = {
  "vm_public_ip" = "54.210.213.165"
}
vpc_output = {
  "private_subnet_id" = [
    "subnet-00911a0a2d79cb072",
    "subnet-0b9f76f5ba5e74a78",
  ]
  "public_subnet_id" = [
    "subnet-06c545bed4226902f",
    "subnet-0b5204e52c51f66a4",
  ]
  "vpc_id" = "vpc-092675b00c845e543"
}
```



Task 5: Debugging and Fixing Terraform Issues

1. נתחיל בפקודות הבאות לבדיקה:

```
terraform validate
```

```
terraform plan -out=plan.tf
```

```
TF_LOG=DEBUG terraform apply
```

2. בדיקת חוקי Security Groups:

הפקודות הבאות מספקות לנו דרך לזהות sg שגויים ולבדוק האם הם תואמים את ההגדרות הנכונות של ה-EC2 וה-ALB

```
terraform state show aws_security_group.alb_sg
```

```
terraform state show aws_security_group.ec2_sg
```

```
terraform plan -target=aws_security_group.alb_sg
```

```
terraform plan -target=aws_security_group.ec2_sg
```

3. בדיקת הרשאות IAM

הרצת הפקודה לבדיקת שגיאות הרשאה:

```
TF_LOG=DEBUG terraform plan
```

אם מתקבלות שגיאות AccessDenied, נוסף את ההרשאות החסרות:

```
aws iam attach-user-policy --user-name TERRAFORM_USER --policy-arn  
arn:aws:iam::aws:policy/AmazonEC2FullAccess
```

```
aws iam attach-user-policy --user-name TERRAFORM_USER --policy-arn  
arn:aws:iam::aws:policy/ElasticLoadBalancingFullAccess
```

4. שגיאות נוספות נפוצות

InvalidClientTokenId – check AWS credentials

AccessDenied- Add missing IAM permissions

MissingRegion- Verifying correct region setting

ValidationError- Checking Security Groups rules

5. הוראות לביצוע debugging

```
export TF_LOG=DEBUG
```

```
export TF_LOG_PATH=./terraform.log
```

```
terraform init -reconfigure
```

```
terraform plan
```

אם עדיין נראה שיש שגיאות אפשר לבדוק את קובץ הלוג המפורט ב terraform.log

